

DOKUMENTASI PBS

PoolTick



Nama	: Nazmi Mustafa
NPM	: 23312213
JobDes	: CMS, USER
Nama Kelompok	: Teluk On Top

Daftar Isi

Daftar Isi	2
CMS.....	3
1. Menginisialisasi aplikasi NestJS (main.ts).	3
2. Sebagai "pengikat" atau orchestrator dari semua komponen (app.module.ts). ...	3
3. Menangani request HTTP yang masuk ke CMS (app.controller.ts)	4
4. Konfigurasi Alamat API	4
5. Bagian Mengambil Data	4
6. Bagian Mengirim/Menyimpan Data	5
7. Bagian Proses Pembelian	6
8. Bagian Penghapusan Data.....	6
9. Halaman Login	7
10. Navigator Header	8
11. Stat Card (Summary Dashboard).....	9
12. Export Laporan Keuangan (Excel)	10
13. Form Identitas Pelanggan	12
14. Form Kelola Katalog Tiket.....	13
15. Modul Katalog Tiket & Kasir	14
16. Modul Riwayat Transaksi.....	15
17. Struktur Proyek: Modularitas & Maintenance	16
18. Optimasi Tabel Log Transaksi	18
19. Optimasi Penomoran Log Transaksi	19
USER.....	20
20. Aplikasi Mobile: Sisi Pengguna (Flutter)	20
21. Struktur Proyek: Arsitektur Modular (Flutter).....	21
22. Halaman Pemesanan & Metode Pembayaran	22

CMS

1. Menginisialisasi aplikasi NestJS (main.ts).

```
1 import { NestFactory } from '@nestjs/core';
2 import { AppModule } from './app.module';
3 import { join } from 'path';
4 import * as express from 'express';
5
6 async function bootstrap() {
7   const app = await NestFactory.create(AppModule);
8
9   app.use(express.static(join(__dirname, '..', 'src', 'views')));
10
11   await app.listen(3001);
12 }
13 bootstrap();
```

Bagian baris nomor 11 “`await app.listen(3001)`” itu saya bedakan dari localhost dari backend. Jadi untuk memanggil CMS yaitu dengan localhost:3001.

Dan untuk bagian baris nomor 9 “`express.static(join(__dirname, '..', 'src', 'views'))`” ini ada bagian untuk memanggil tampilan interface dari CMS.

2. Sebagai "pengikat" atau orchestrator dari semua komponen (app.module.ts).

```
1 import { Module } from '@nestjs/common';
2 import { AppController } from './app.controller';
3 import { AppService } from './app.service';
4
5 @Module({
6   imports: [],
7   controllers: [AppController],
8   providers: [AppService],
9 })
10 export class AppModule {}
11
```

File ini digunakan sebagai Root Module atau module utama dalam aplikasi CMS Pooltick. Fungsinya sebagai pusat registrasi komponen utama aplikasi agar dapat dikelola NestJs Container.

3. Menangani request HTTP yang masuk ke CMS (app.controller.ts)

```
1 import { Controller, Get } from '@nestjs/common';
2 import axios from 'axios';
3
4 @Controller()
5 export class AppController {
6
7   @Get('tickets')
8   async getTickets() {
9     const res = await axios.get('http://localhost:3000/tickets');
10    return res.data;
11  }
12 }
```

Ini adalah file app controller untuk di bagian baris nomor 7 “@Get('tickets')” ini adalah endpoint untuk CMS mengambil data tiket.

Dan untuk bagian baris nomor 9 “await axios.get('http://localhost:3000/tickets')” bahwasan ini CMS bertindak sebagai client yang melakukan request ke API/Backend.

4. Konfigurasi Alamat API

```
1 const API_TICKETS = 'http://localhost:3000/tickets';
2 const API_TRANSACTIONS = 'http://localhost:3000/transactions';
```

Ini adalah code dari folder scr-views-index.html. Code ini variabel dasar yang menentukan kemana CMS harus meminta data. Semua CRUD akan lari ke URL ini di localhost:3000 yaitu bagian API/Backend.

5. Bagian Mengambil Data

```
1 const [tickets, transactions] = await Promise.all([
2   fetch(API_TICKETS).then(r => r.json()),
3   fetch(API_TRANSACTIONS).then(r => r.json())
4 ]);
```

Code ini menggunakan fetch yang berguna sebagai menarik data mentah dari API. Karena CMS tidak memiliki database sendiri dan sangat bergantung data yang dikirim dari API. Cara

kerjanya:

- **Asynchronous Request (await):** CMS menggunakan perintah await yang artinya sistem akan menunggu sampai data dari API benar-benar terkirim sebelum mencoba menampilkannya. Ini mencegah aplikasi error karena mencoba menampilkan data yang belum ada.
- **Parallel Fetching (Promise.all):** Ini teknik tingkat tinggi. CMS mengambil data Tiket dan data Transaksi secara bersamaan (paralel). Ini membuat aplikasi terasa jauh lebih cepat daripada mengambil datanya satu per satu.
- **Data Synchronization:** Setelah data mentah (JSON) diterima, CMS langsung memprosesnya ke fungsi renderTickets dan renderTransactions untuk mengubah kode-kode komputer menjadi tabel yang bisa dibaca manusia.

6. Bagian Mengirim/Menyimpan Data

```
1  const method = id ? 'PATCH' : 'POST';
2  const url = id ? `${API_TICKETS}/${id}` : API_TICKETS;
3
4  await fetch(url, {
5    method, headers: {'Content-Type': 'application/json'},
6    body: JSON.stringify({ name, price })
7  });
8  resetForm();
9  loadData();
10 }
```

Code di baris ini menentukan apakah CMS sedang dalam menambahkan tiket baru yaitu dengan POST atau mengedit tiket lama yaitu PATCH. Data dikirim dalam format JSON ke API/Backend.

Cara kerja:

- **Dynamic Method Selection:** Kode kamu sangat efisien karena menggunakan satu fungsi untuk dua kegunaan.
 - Jika ada ID, CMS menggunakan metode PATCH (Update).
 - Jika ID kosong, CMS menggunakan metode POST (Tambah Baru).
- **Data Encapsulation:** Data seperti Nama Tiket dan Harga dibungkus ke dalam format JSON.stringify. Ini adalah standar industri untuk pengiriman data antar aplikasi (dari CMS ke API).

- **Header Configuration:** Penggunaan 'Content-Type': 'application/json' memberi tahu Backend: "Woi Backend, saya kirim paket berupa data JSON, tolong dibongkar sesuai format ya!".

7. Bagian Proses Pembelian



```

1 for (let i = 0; i < qty; i++) {
2   await fetch(API_TRANSACTIONS, {
3     method: 'POST', headers: {'Content-Type': 'application/json'},
4     body: JSON.stringify({
5       ticketId: t.id, name: buyer, price: t.price, createdAt: new Date().toISOString()
6     })
7   });
8 }

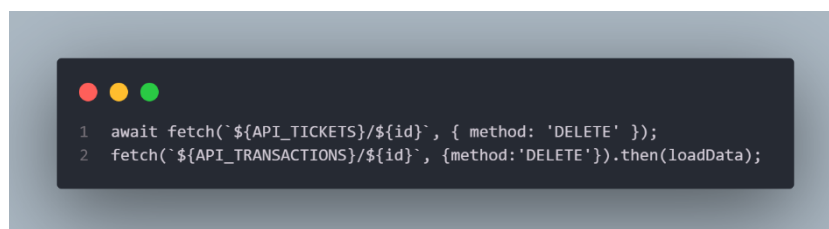
```

Code ini adalah inti dari sisi USER/PELANGGAN. Setiap tombol “Checkout” ditekan, CMS akan mengirimkan data transaksi satu per satu ke database API/Backend.

Cara kerja:

- **Looping Transaksi:** Kode menggunakan `for (let i = 0; i < qty; i++)`. Ini berarti jika user membeli 3 tiket sekaligus, CMS akan melakukan 3 kali pengiriman data (request) ke API secara otomatis.
- **Payload (Data yang dikirim):** CMS membungkus data berupa `ticketId` (ID tiket), `name` (Nama pembeli), `price` (Harga saat itu), dan `createdAt` (Waktu transaksi).
- **Metode HTTP POST:** CMS menggunakan metode POST ke endpoint `API_TRANSACTIONS`. Ini memberi instruksi ke Backend: "Tolong buat satu catatan transaksi baru dengan data ini."

8. Bagian Penghapusan Data



```

1 await fetch(`${API_TICKETS}/${id}`, { method: 'DELETE' });
2 fetch(`${API_TRANSACTIONS}/${id}`, { method: 'DELETE' }).then(loadData);

```

Untuk bagian code ini instruksi langsung dari backend untuk menghapus data tertentu berdasarkan ID uniknya.

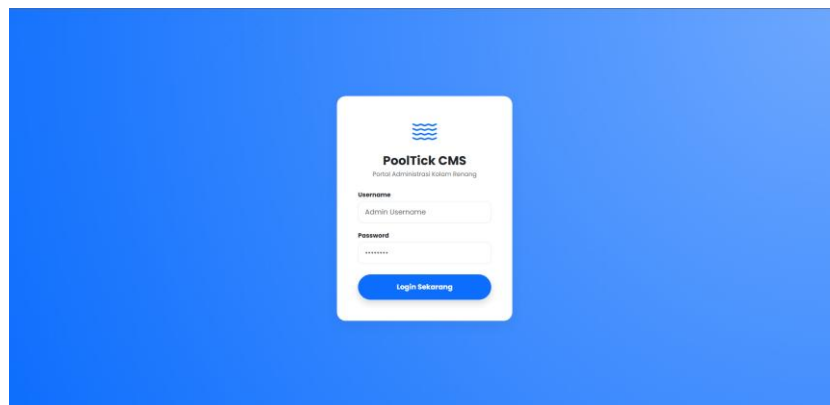
Cara kerja:

- **Metode HTTP DELETE:** CMS mengirimkan request dengan metode DELETE yang diikuti oleh ID spesifik di akhir URL-nya (contoh:

localhost:3000/tickets/5). Ini memastikan hanya data dengan ID tersebut yang terhapus.

- **Validasi Integritas:** Perhatikan kode if (`trx.some(t => t.ticketId == id)`). Ini adalah fitur cerdas di CMS kamu. Sebelum menghapus tiket, CMS mengecek ke API: "Apakah tiket ini pernah dipakai transaksi?". Jika iya, penghapusan ditolak agar laporan keuangan tidak error (data tiket hilang tapi riwayat transaksi masih ada).
- **Refresh Otomatis:** Setelah perintah DELETE berhasil direspon oleh API, CMS langsung menjalankan `loadData()`. Ini sinkronisasi agar tampilan di layar langsung berubah tanpa user harus menekan tombol refresh browser (F5).

9. Halaman Login



Halaman ini adalah pintu masuk utama untuk Admin sebelum dapat mengelola data.

- **Visual Layout:** Menggunakan teknik centered-card dengan latar belakang radial-gradient biru. Hal ini bertujuan untuk memfokuskan perhatian pengguna pada form input.
- **Elemen UI:**
 - Input Field: Dibuat dengan border-radius: 12px untuk kesan modern.
 - Action Button: Tombol "Login Sekarang" menggunakan rounded-pill dengan efek shadow agar terlihat clickable.
- **UX Experience:** Sistem menggunakan validasi sederhana. Jika login berhasil, halaman login akan disembunyikan (`display: none`) dan dashboard akan dimunculkan tanpa perlu memuat ulang browser (Single Page Experience).

```
1 function login() {
2   // Ambil nilai dari input username dan password
3   const user = document.getElementById('username').value;
4   const pass = document.getElementById('password').value;
5
6   // Cek sederhana (Username: 1, Password: 1)
7   if (user === "1" && pass === "1") {
8     document.getElementById('login-page').style.display = "none";
9     document.getElementById('dashboard').style.display = "block";
10    loadData(); // Jalankan Load data otomatis setelah Login berhasil
11  } else {
12    alert("Login Gagal! Username/Pass: 1");
13  }
14 }
```

- Data Retrieval:

“const user = document.getElementById('username').value;”

Program menggunakan DOM Manipulation untuk mengambil nilai tekstual yang diketikkan user secara real-time.

- Conditional Logic (Hardcoded Validation):

Sistem menggunakan pengecekan berbasis kondisi

if (user === "1" && pass === "1")

Pada tahap pengembangan ini, kredensial masih bersifat statis (Hardcoded) untuk keperluan pengujian lokal, namun struktur fungsi sudah siap untuk diintegrasikan dengan API Authentication.

- State Transition:

Jika validasi berhasil, sistem melakukan dua aksi sekaligus:

1. Manipulasi DOM: Mengubah properti CSS `display` dari `none` ke `block` (dan sebaliknya) untuk memindahkan user dari layar login ke dashboard tanpa memuat ulang halaman (No Page Refresh).
2. Data Initialization: Menjalankan fungsi `loadData()`. Ini memastikan bahwa begitu admin masuk, data tiket dan transaksi terbaru langsung ditarik dari API Backend.

10. Navigator Header



Bagian Header merupakan elemen navigasi utama yang bersifat *persistent* (tetap muncul) untuk memberikan identitas sistem dan akses cepat terhadap pengaturan akun pengguna.

1. Analisis Visual & UI (HTML & CSS)

- Identitas Brand (navbar-brand):
 - Menggunakan kombinasi ikon ombak (bi-water) dan teks "PoolTick" dengan font tebal (fw-bold).

- Warna: Menggunakan warna biru primer (`text-primary`) untuk memperkuat branding aplikasi yang berkaitan dengan fasilitas air.
- User Profile Dropdown:
 - Di sisi kanan, terdapat komponen dropdown yang dibungkus dalam tombol `btn-light` berbentuk kapsul (`rounded-pill`).
 - Menggunakan ikon profil (`bi-person-circle`) dan label "Admin" untuk menunjukkan identitas pengguna yang sedang aktif.
- Styling & Glassmorphism:
 - CSS Class: `sticky-top shadow-sm`.
 - Efek: Navbar menggunakan properti `backdrop-filter: blur(10px)` (efek kaca buram) dan latar belakang semi-transparan. Hal ini memastikan konten di bawah navbar tetap terlihat samar saat di-scroll, menciptakan kesan modern dan mewah.

11. Stat Card (Summary Dashboard)



Komponen ini berfungsi sebagai indikator performa utama (*Key Performance Indicator*) yang menyajikan akumulasi finansial dari seluruh transaksi yang terjadi.

1. Analisis Visual & Desain (HTML & CSS)

- Warna & Psikologi (stat-card):
 - Menggunakan gradien linear: `linear-gradient(135deg, var(--primary), #0a58ca)`.
 - Analisis: Warna biru tua memberikan kesan kepercayaan dan stabilitas. Teknik gradien digunakan untuk memberikan efek kedalaman agar kartu terlihat lebih menonjol dibandingkan elemen lain.
- Tipografi & Kontras:
 - Teks "Total Pendapatan Hari Ini" menggunakan `text-uppercase` dengan `opacity-75` agar fokus Admin langsung tertuju pada angka nominalnya yang lebih terang.
 - Angka nominal menggunakan class `display-4` untuk memastikan keterbacaan maksimal (*high visibility*).

- Dekorasi Ikon:
 - Menggunakan ikon dompet ([bi-wallet2](#)) dengan ukuran sangat besar ([display-1](#)) dan [opacity-25](#).
 - Tujuan UX: Memberikan petunjuk visual mengenai kategori kartu (Keuangan) tanpa mengalihkan perhatian dari data teks utama.

2. Analisis Logika Data (JavaScript)

Angka yang muncul di sini bukan angka mati, melainkan hasil perhitungan dinamis.

- Fungsi Perhitungan (`renderTransactions`): Di dalam file `script.js` (sebelumnya di bagian `render`), terdapat logika pemrosesan data:



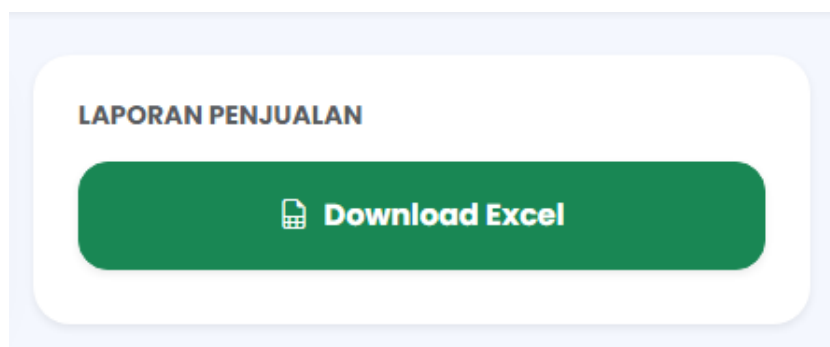
```

1 let total = 0;
2 transactions.map(i => {
3   total += i.price;
4   // ...
5 });
6 document.getElementById('total').innerText = rupiah(total);

```

- Mekanisme Kerja:
 1. Sistem mengambil seluruh data dari API.
 2. Variabel `total` akan menjumlahkan setiap harga (`price`) dari semua objek transaksi yang ada.
 3. Hasil akhir diformat menjadi mata uang Rupiah menggunakan fungsi `rupiah()` sebelum ditampilkan ke elemen HTML dengan ID `total`.

12. Export Laporan Keuangan (Excel)



Fitur ini memungkinkan Admin untuk mengunduh seluruh data transaksi yang tersimpan di API Backend ke dalam format spreadsheet (.xlsx) yang rapi dan siap cetak.

1. Alur Kerja Pengumpulan Data (Multi-Fetch)

Sebelum file dibuat, sistem harus mengumpulkan "bahan baku" datanya dulu.

- Kode: `const [dataTrx, dataTickets] = await Promise.all([...])`

- Penjelasan: Sistem menarik dua data berbeda sekaligus: data Transaksi (siapa yang beli) dan data Tiket (apa yang dibeli). Teknik [Promise.all](#) digunakan agar proses penarikan data lebih cepat daripada menariknya satu-satu.

2. Transformasi Data (Data Mapping)

Data mentah dari API biasanya sulit dibaca karena hanya berisi angka/ID. Di sini sistem melakukan "penerjemahan".

- Kode: `const tiket = dataTickets.find(t => t.id === trx.ticketId);`
- Penjelasan: Sistem mencocokkan `ticketId` di setiap transaksi dengan daftar tiket yang ada.
 - Hasilnya: Di file Excel nanti yang muncul adalah tulisan "Tiket Terusan", bukan cuma angka "1". Ini meningkatkan kualitas laporan secara signifikan.

3. Penambahan Baris Total (Kalkulasi Otomatis)

Ini bagian yang kamu tanyakan tadi, kenapa ada `reportData.push`.

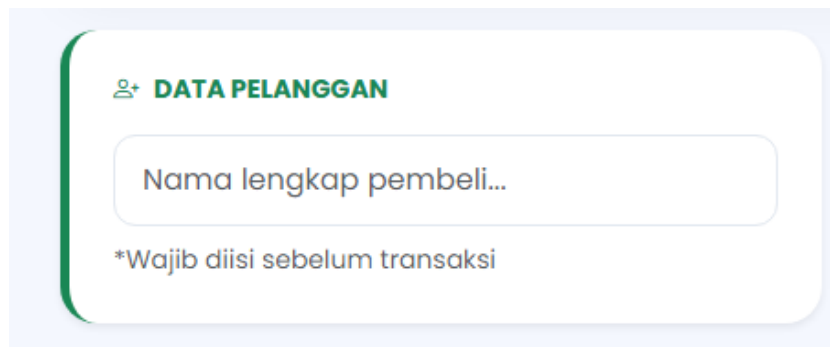
- Kode: `reportData.push({ "Nama Pembeli": "TOTAL PENDAPATAN", "Harga (Rp)": totalKeseluruhan });`
- Penjelasan: * Tujuan: Agar Admin tidak perlu menjumlahkan manual di Excel.
 - Cara Kerja: Sambil memproses data, sistem menghitung `totalKeseluruhan`. Di akhir proses, sistem menyisipkan satu baris tambahan di posisi paling bawah Excel yang berisi teks "TOTAL PENDAPATAN" dan angka total uangnya.

4. Teknis Pembuatan File (Library XLSX)

Sistem menggunakan library SheetJS untuk membungkus data menjadi file fisik.

- Worksheet ([ws](#)): Mengubah array objek JavaScript menjadi lembaran kerja Excel.
- Workbook ([wb](#)): Menciptakan file Excel baru yang bisa berisi beberapa lembar kerja.
- WriteFile: Baris terakhir `XLSX.writeFile(...)` adalah perintah yang memicu browser untuk otomatis mengunduh file ke komputer Admin dengan nama file unik (menggunakan `new Date().getTime()`).

13. Form Identitas Pelanggan



Dalam konteks operasional kolam renang, CMS ini berfungsi sebagai sistem kasir (Point of Sales) untuk melayani pembelian tiket secara langsung (on-the-spot). Berikut adalah penjelasan teknis dan fungsionalnya:

1. Peran Admin sebagai Operator

Sistem ini dirancang bukan untuk diisi oleh pelanggan, melainkan oleh Admin/Petugas Loker.

- Proses: Ketika pelanggan datang ke loket untuk membeli tiket secara tunai (cash), Admin akan menanyakan nama pelanggan dan tiket apa yang ingin dibeli.
- Input Data: Admin mengetikkan nama pada kolom "Data Pelanggan". Ini memastikan bahwa meskipun pembayaran dilakukan secara tunai, setiap lembar tiket yang terjual tetap memiliki catatan kepemilikan yang jelas di database.

2. Metode Pembayaran Cash (Tunai)

Karena ini adalah sistem manajemen internal kolam renang (CMS), proses pembayaran dilakukan secara *offline*.

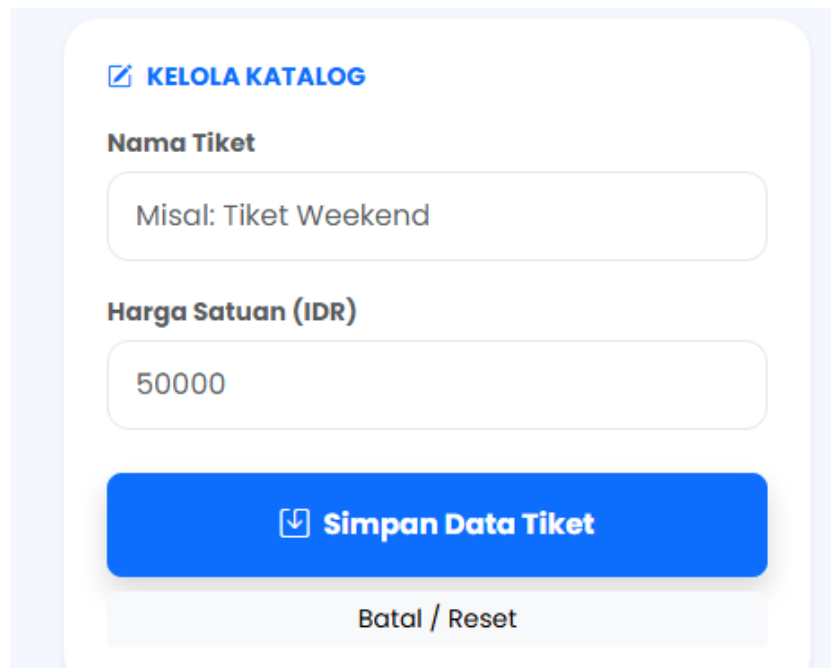
- Mekanisme: Setelah Admin menekan tombol "Checkout Terpilih", sistem menganggap pembayaran cash telah diterima oleh Admin.
- Pencatatan: Secara otomatis, sistem merekam transaksi tersebut ke dalam API (port 3000) sebagai transaksi sukses. Nominal uang yang diterima Admin akan langsung terakumulasi pada bagian "Total Pendapatan Hari Ini".

3. Keuntungan untuk Pengelola

Dengan admin yang mengisi data secara manual untuk setiap pembelian cash:

- Transparansi: Pemilik kolam renang bisa memantau apakah jumlah uang tunai di laci kasir sama dengan angka yang tertera di Total Pendapatan pada CMS.
- Keamanan: Mencegah adanya tiket yang terjual tanpa tercatat (mengurangi risiko kecurangan/kebocoran dana).
- Laporan Akurat: Saat akhir hari, Admin tinggal klik "Download Excel" untuk menyerahkan laporan penjualan harian kepada pemilik tanpa harus menghitung ulang nota manual.

14. Form Kelola Katalog Tiket



KELOLA KATALOG

Nama Tiket

Misal: Tiket Weekend

Harga Satuan (IDR)

50000

Simpan Data Tiket

Batal / Reset

Panel ini merupakan pusat manajemen data master yang memungkinkan Admin untuk menambah atau memperbarui informasi tiket secara dinamis.

1. Komponen Antarmuka (UI)

- Input Nama Tiket: Field teks untuk mendefinisikan kategori tiket (contoh: *Tiket Weekend*, *Tiket Anak-anak*, *VIP*).
- Input Harga Satuan: Menggunakan atribut `type="number"`. Secara UX, ini mencegah Admin memasukkan karakter selain angka, sehingga data yang dikirim ke database selalu berupa nilai numerik yang valid untuk perhitungan.
- Tombol Simpan (Primary Action): Menggunakan warna biru (`btn-primary`) untuk menunjukkan aksi utama. Tombol ini memiliki ikon `bi-save` untuk memperkuat pesan visual.
- Tombol Batal / Reset: Tombol sekunder yang berfungsi membersihkan formulir jika Admin salah input atau ingin membatalkan mode edit.

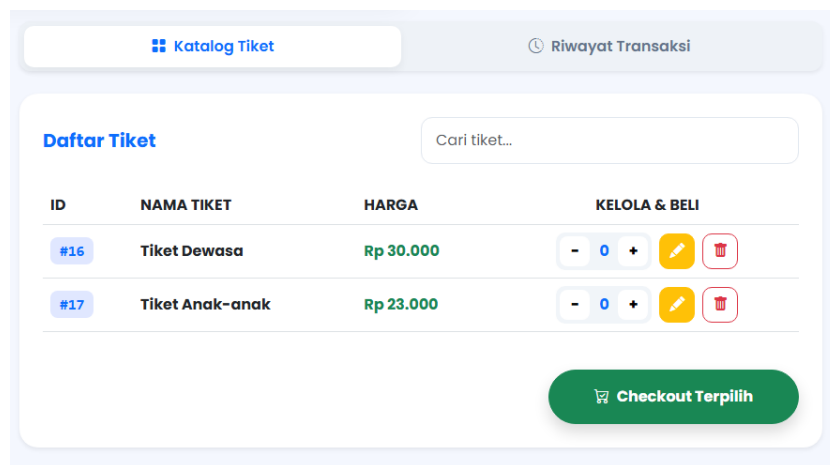
2. Analisis Logika "Dual-Mode" (JavaScript)

Salah satu kecanggihan form ini adalah kemampuannya berubah fungsi tanpa harus pindah halaman.

- Mode Create (Tambah): Secara default, form ini kosong. Ketika tombol ditekan, fungsi `submitTiket()` akan menjalankan perintah HTTP POST ke backend untuk membuat baris data baru.
- Mode Update (Edit): Ketika Admin mengklik ikon pensil di tabel tiket, fungsi `editTiket()` dipanggil.
 - Perubahan State: Data tiket lama akan "dilempar" masuk ke kotak input.
 - Visual Feedback: Tombol akan otomatis berubah warna menjadi kuning dan teksnya berubah menjadi "Perbarui Data Tiket".

- Logika Backend: Saat disimpan, sistem akan menjalankan perintah HTTP PATCH ke ID spesifik tiket tersebut.
3. Fitur Keamanan & Validasi
- Validation Check: Di dalam kode `script.js`, terdapat pengecekan `if (!name || !price)`. Jika Admin lupa mengisi salah satu field, sistem akan memunculkan peringatan (alert) dan menolak pengiriman data. Hal ini menjamin integritas data di database backend (port 3000).
 - Reset Logic: Setelah data berhasil disimpan atau diperbarui, fungsi `resetForm()` akan dijalankan secara otomatis untuk mengosongkan input dan mengembalikan tombol ke kondisi awal (warna biru).

15. Modul Katalog Tiket & Kasir



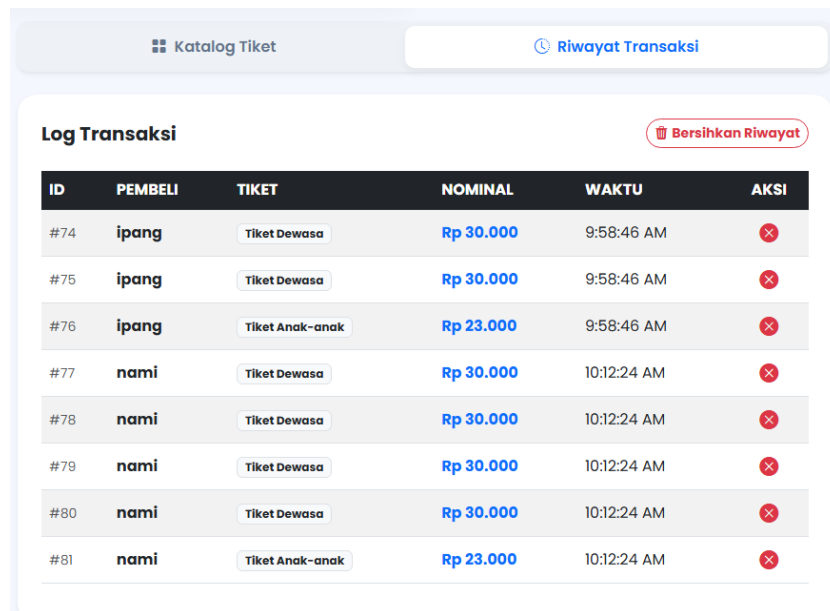
1. Fitur Pencarian Real-Time (searchTicket)
 - Implementasi: Input teks dengan atribut `onkeyup="filterTable()"`.
 - Cara Kerja: Sistem akan memfilter baris tabel secara otomatis saat Admin mengetikkan nama tiket.
 - Tujuan UX: Mempercepat layanan jika daftar tiket sudah sangat banyak (misal: tiket rombongan, tiket event, tiket promo). Admin tidak perlu melakukan *scroll* panjang, cukup ketik keyword (contoh: "Dewasa").
2. Komponen Baris Tabel (Item Tiket)

Setiap baris tiket memiliki elemen kontrol yang sangat lengkap:

- ID Badge: Menampilkan ID unik tiket (contoh: #16) dengan font monospace agar mudah dibedakan secara visual.
- Interactive QTY Box: * Tombol plus (+) dan minus (-) memudahkan Admin menambah jumlah tiket tanpa keyboard.
 - Angka jumlah tiket (`qtyMap`) ditampilkan di tengah dengan warna biru agar terlihat jelas.
- Action Buttons:

- Ikon Pensil (Kuning): Memicu fungsi `editTiket()` untuk mengubah harga atau nama tiket tersebut.
 - Ikon Sampah (Merah): Memicu fungsi `hapusTiket()`. Sistem memiliki proteksi; jika tiket tersebut sudah pernah ada transaksinya, sistem akan menolak penghapusan untuk menjaga integritas data laporan.
3. Aksi Checkout Terpilih (beliSemua)
- Visual: Tombol besar berwarna hijau (`btn-success`) dengan ikon keranjang belanja.
 - Logika Bisnis:
 - Tombol ini hanya akan memproses tiket yang jumlahnya (QTY) lebih dari 0.
 - Setelah diklik, sistem akan melakukan pengiriman data ke API secara massal dan mengosongkan keranjang belanja kembali ke angka 0.

16. Modul Riwayat Transaksi



ID	PEMBELI	TIKET	NOMINAL	WAKTU	AKSI
#74	ipang	Tiket Dewasa	Rp 30.000	9:58:46 AM	✖
#75	ipang	Tiket Dewasa	Rp 30.000	9:58:46 AM	✖
#76	ipang	Tiket Anak-anak	Rp 23.000	9:58:46 AM	✖
#77	nami	Tiket Dewasa	Rp 30.000	10:12:24 AM	✖
#78	nami	Tiket Dewasa	Rp 30.000	10:12:24 AM	✖
#79	nami	Tiket Dewasa	Rp 30.000	10:12:24 AM	✖
#80	nami	Tiket Dewasa	Rp 30.000	10:12:24 AM	✖
#81	nami	Tiket Anak-anak	Rp 23.000	10:12:24 AM	✖

Modul ini berfungsi sebagai catatan digital otomatis yang merekam setiap aktivitas penjualan tiket yang telah dilakukan oleh Admin secara kronologis.

1. Navigasi & Kontrol Header

- Tab Switcher: Bagian ini aktif saat tab Riwayat Transaksi dipilih. Antarmuka akan beralih dari daftar katalog ke daftar catatan penjualan menggunakan sistem *tabbing* Bootstrap.
- Tombol Bersihkan Riwayat:
 - Visual: Tombol berwarna merah (`outline-danger`) dengan ikon sampah (`bi-trash3-fill`).
 - Fungsi Teknis (`hapusSemuaTransaksi`): Mengosongkan seluruh database transaksi di backend. Fitur ini dilengkapi dengan dialog

konfirmasi (confirm) untuk mencegah penghapusan massal data secara tidak sengaja.

2. Struktur Tabel Log

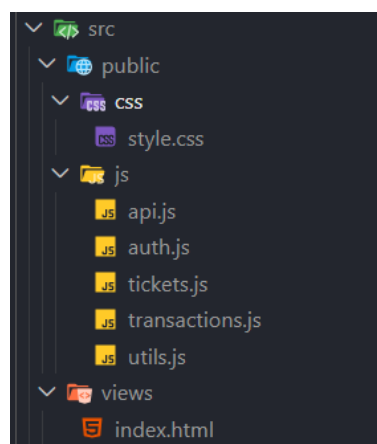
Tabel ini menyajikan data mentah dari database dalam format yang mudah dibaca:

- ID Transaksi: Menggunakan tanda pagar (contoh: #74) sebagai identitas unik setiap tiket yang terjual.
- Nama Pembeli: Menampilkan nama pelanggan yang diinput oleh Admin pada saat checkout. Nama dicetak tebal (fw-bold) untuk memudahkan identifikasi subjek transaksi.
- Label Tiket: Menggunakan komponen *Badge* abu-abu. Secara teknis, kolom ini melakukan pencarian relasional (lookup) ke data katalog tiket untuk menampilkan nama tiket berdasarkan ID-nya.
- Nominal: Ditampilkan dengan warna biru (text-primary) untuk membedakannya dengan teks lain, menunjukkan nilai uang dari transaksi tersebut.
- Waktu (Timestamp): Menampilkan jam transaksi dengan format lokal (toLocaleTimeString). Hal ini membantu Admin melacak waktu tersibuk di kolam renang.

3. Aksi Pembatalan (Delete Single Row)

- Ikon X Merah: Tombol aksi untuk menghapus satu baris transaksi tertentu.
- Kegunaan: Sangat berguna jika Admin melakukan kesalahan input (misal: salah ketik nama atau salah pilih tiket) dan ingin menghapus record tersebut agar total pendapatan kembali akurat.

17. Struktur Proyek: Modularitas & Maintenance



Memisahkan logika program ke dalam file-file spesifik, sistem menjadi lebih terorganisir dan mudah untuk maintenance.

1. Folder public/css

- `style.css`: Berisi seluruh aturan desain (UI), variabel warna, dan *layouting* dashboard.
- Alasan Maintenance: Jika admin ingin mengubah warna tema aplikasi (misal dari biru ke hijau), perubahan hanya dilakukan pada file ini tanpa menyentuh logika program.

2. Folder public/js (Modular Logic)

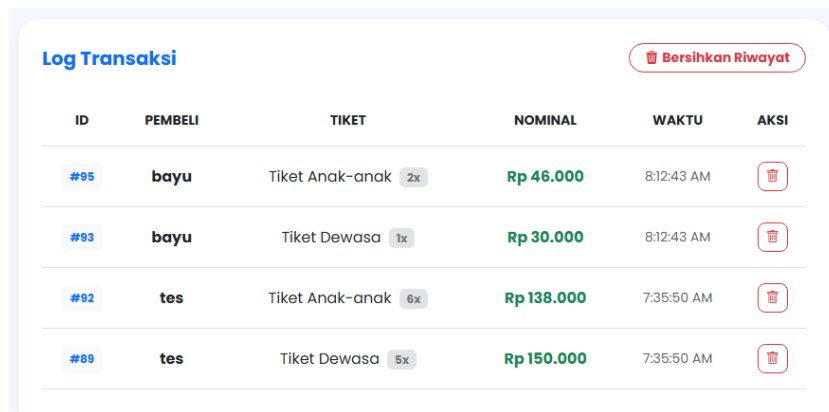
Di sini aplikasi dipisahkan berdasarkan tugasnya:

- `api.js`:
 - Peran: Menyimpan konfigurasi URL (port 3000) dan fungsi dasar untuk koneksi ke server.
 - Kemudahan Maintenance: Jika alamat server berubah (misal dari localhost ke domain asli), Admin cukup mengubah satu baris di file ini.
- `auth.js`:
 - Peran: Khusus menangani alur login dan logout.
 - Kemudahan Maintenance: Jika ingin mengganti sistem login ke yang lebih canggih (seperti JWT atau OAuth), kodenya tidak akan bercampur dengan kode katalog.
- `tickets.js`:
 - Peran: Menangani manajemen katalog (Tambah, Edit, Hapus, dan Tampilan Tiket).
- `transactions.js`:
 - Peran: Menangani seluruh alur transaksi, perhitungan total pendapatan, riwayat, dan fitur ekspor Excel.
- `utils.js`:
 - Peran: Berisi fungsi pembantu (helper) seperti format mata uang Rupiah atau format tanggal.
 - Kemudahan Maintenance: Fungsi ini bisa dipanggil berulang kali di file lain (reusable), sehingga kode lebih ringkas.

3. Folder views

- `index.html`:
 - Peran: Hanya sebagai kerangka (skeleton) aplikasi.
 - Kemudahan Maintenance: File ini jadi sangat bersih dan hanya berisi tag-tag HTML. Tidak ada lagi tumpukan ratusan baris skrip JavaScript di dalamnya.

18. Optimasi Tabel Log Transaksi



ID	PEMBELI	TIKET	NOMINAL	WAKTU	AKSI
#95	bayu	Tiket Anak-anak 2x	Rp 46.000	8:12:43 AM	
#93	bayu	Tiket Dewasa 1x	Rp 30.000	8:12:43 AM	
#92	tes	Tiket Anak-anak 6x	Rp 138.000	7:35:50 AM	
#89	tes	Tiket Dewasa 5x	Rp 150.000	7:35:50 AM	

Bagian ini menjelaskan pembaruan desain pada modul riwayat transaksi untuk meningkatkan keterbacaan data dan efisiensi ruang antarmuka.

1. Komponen Visual Baru

- **Badge Quantity Dinamis:**
 - **Fitur:** Menambahkan label jumlah beli (contoh: 2x, 5x) di samping nama tiket menggunakan badge berwarna abu-abu muda.
 - **Fungsi UX:** Memberikan informasi volume pembelian dalam satu baris yang ringkas tanpa perlu kolom tambahan, sehingga tabel tetap terlihat lega.
- **Tipografi Berjenjang:**
 - **Nama Pembeli:** Menggunakan font tebal (fw-bold) dengan warna hitam pekat untuk memperjelas identitas subjek transaksi.
 - **Nominal Harga:** Menggunakan warna hijau (text-success) yang kontras untuk memisahkan data keuangan dengan data teks lainnya.
- **Aksi Hapus yang Disederhanakan:**
 - **Ikon:** Menggunakan ikon tempat sampah dalam kotak merah yang lembut.
 - **Fungsi:** Memberikan ruang klik (*hitbox*) yang lebih pas bagi admin untuk melakukan pembatalan transaksi secara individu.

19. Optimasi Penomoran Log Transaksi

Log Transaksi					
Bersihkan Riwayat					
NO	PEMBELI	TIKET	NOMINAL	WAKTU	AKSI
1	gg	Tiket Anak-anak 1x	Rp 24.000	10:43:25 PM	
2	gg	Tiket Dewasa 2x	Rp 60.000	10:43:25 PM	
3	bu warto	Tiket Anak-anak 2x	Rp 48.000	9:08:58 AM	
4	bu warto	Tiket Dewasa 3x	Rp 90.000	9:08:58 AM	

Bagian ini menjelaskan perubahan format penomoran pada tabel Log Transaksi dari yang sebelumnya menampilkan ID database menjadi nomor urut dinamis.

1. Konsep Pemisahan Data & Tampilan

Sistem menerapkan prinsip pemisahan antara data backend dan representasi visual frontend:

- Sisi Backend (Port 3000): Tetap menggunakan properti id unik bawaan database untuk setiap record transaksi. ID ini tidak diubah agar relasi data dan fungsi hapus data tunggal (DELETE /transactions/:id) tetap berjalan aman tanpa merusak struktur database.
- Sisi Frontend (CMS Web): Kolom pertama diubah menjadi NO (Nomor Urut). Sistem tidak lagi menampilkan nilai i.id mentah, melainkan menghitung urutan index baris secara dinamis saat data dirender ke tabel HTML.

2. Analisis Logika JavaScript

Perubahan ini diimplementasikan dengan memanfaatkan parameter index pada perulangan fungsi .map() di dalam renderTransactions:

JavaScript

```
1 <!-- GANTI i.id MENJADI index + 1 UNTUK TAMPILAN NOMOR URUT -->
2 <td class="py-3 text-center">
3   <span class="badge bg-light text-primary fw-bold p-2">${index + 1}</span>
4 </td>
```

3. Keuntungan Fungsional (UI/UX)

- Kerapian Data: Tabel terlihat lebih rapi dan konsisten karena nomor urut selalu dimulai dari angka 1 hingga jumlah baris terakhir, tidak peduli jika ada data di tengah yang sudah dihapus.

- Integritas Sistem: Tombol aksi hapus (ikon tempat sampah merah) tetap mengirimkan i.id database yang asli ke fungsi `hapusTransaksi(id)`, sehingga sistem tetap tahu persis baris data mana yang harus dihapus dari server backend.

USER

20. Aplikasi Mobile: Sisi Pengguna (Flutter)

1. Arsitektur Komponen & State Management

- Perubahan State (StatefulWidget):
 - Aplikasi diubah dari StatelessWidget menjadi StatefulWidget (`_HomePageState`).
 - Alasan Teknis: Perubahan ini krusial karena aplikasi sekarang harus mengelola state dinamis secara *real-time*, seperti menangkap teks dari input form menggunakan `TextEditingController` `buyerController`.
- Asynchronous Data Fetching (FutureBuilder):
 - Aplikasi memanfaatkan widget `FutureBuilder` untuk menangani siklus hidup pengambilan data dari API secara *asynchronous* tanpa memblokir proses rendering UI utama (*non-blocking UI*).

2. Integrasi Komunikasi API (HTTP Client)

Aplikasi mobile berkomunikasi langsung dengan backend (port 3000) menggunakan package `http`:

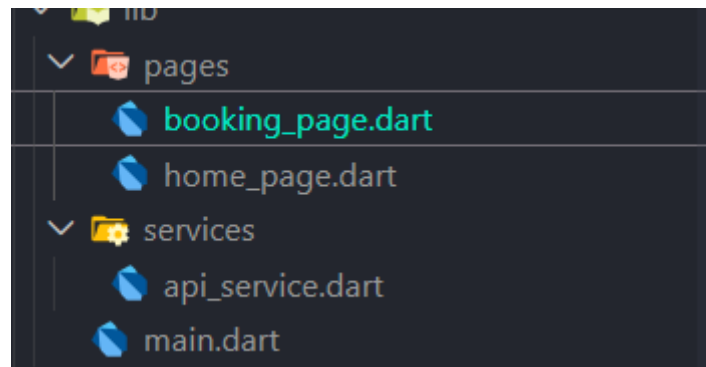
- Pengambilan Data Tiket (`fetchTickets`):
 - Metode: HTTP GET ke `http://localhost:3000/tickets`.
 - Proses: Mengambil data katalog tiket terbaru dari database backend, lalu melakukan decoding format JSON menjadi objek `List Dart` untuk dirender ke dalam widget `list`.
- Proses Pembelian Tiket (`beliTiket`):
 - Metode: HTTP POST ke `http://localhost:3000/transactions`.
 - Validasi: Sistem mengecek `buyerController.text.isEmpty`. Jika nama pembeli kosong, sistem memicu `SnackBar` bertuliskan "*Isi nama pembeli dulu*" sebagai bentuk *error prevention*.
 - Payload Data: Mengirim data `ticketId` dan `name` dalam format JSON. Jika server merespons dengan status code 200 atau 201 (Sukses), field input nama akan dibersihkan otomatis via `buyerController.clear()`.

3. Analisis dan Pembaruan Antarmuka (UI/UX)

Struktur UI dirombak dari `ListView.builder` sederhana menjadi tata letak linier `ListView` yang lebih kaya informasi:

- Header & Media Informasi (Baris Baru):
 - Menambahkan judul deskriptif "*Kolam Renang PoolTick*" beserta teks promosi.
 - Mengintegrasikan gambar representatif eksternal menggunakan `Image.network("https://picsum.photos/500/250")` yang dibungkus dengan `ClipRRect` agar memiliki sudut membulat (*rounded corner*), meningkatkan estetika visual aplikasi.
- Form Input Nama Pembeli (TextField):
 - Diimplementasikan menggunakan widget `TextField` yang terikat dengan `buyerController`, menggunakan dekorasi `OutlineInputBorder()` agar terlihat modern dan kontras.
- Katalog Tiket Dinamis (Spread Operator):
 - Menggunakan operator `...data.map((item) { ... }).toList()` untuk merender deretan Card tiket langsung di bawah komponen form input dalam satu aliran *scroll* yang mulus.

21. Struktur Proyek: Arsitektur Modular (Flutter)



Aplikasi mobile PoolTick dirancang ulang menggunakan pendekatan modular guna mempermudah proses pemeliharaan (*maintenance*), pengetesan kode (*unit testing*), serta skalabilitas aplikasi di masa mendatang.

1. Berkas Utama (main.dart)

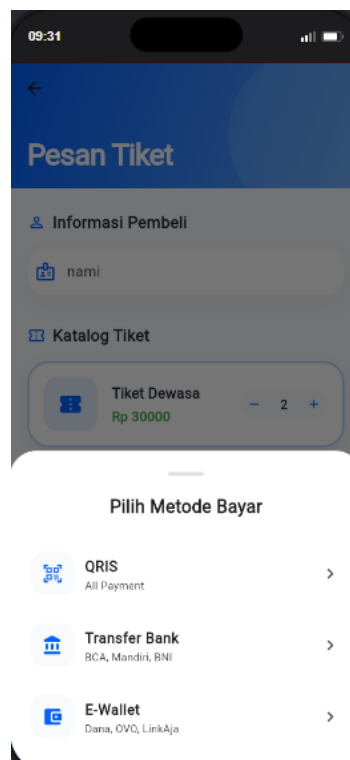
- Peran: Sebagai *entry point* atau gerbang awal jalannya aplikasi Flutter.
- Alasan Maintenance: File ini sekarang menjadi sangat bersih. Fungsinya hanya berfokus untuk inisialisasi awal aplikasi (`runApp`) dan menentukan halaman pertama yang akan dibuka (`home: HomePage()`). Tidak ada lagi tumpukan fungsi API atau UI di sini.

2. Direktori lib/pages (Presentation Layer)

Folder ini dikhususkan hanya untuk mengatur komponen tampilan (UI/UX) yang dilihat oleh pengguna:

- `home_page.dart`:
 - Peran: Menampilkan halaman utama berupa banner promosi kolam renang dan daftar katalog tiket dinamis.
 - Kemudahan Maintenance: Jika kamu ingin mengubah tata letak kartu, mengganti font, atau mendesain ulang banner gambar, kamu hanya perlu mengedit file ini tanpa takut merusak fungsi transaksi.
 - `booking_page.dart`:
 - Peran: Mengelola halaman formulir pemesanan, tempat pelanggan mengonfirmasi data identitas dan memproses *checkout* tiket.
 - Kemudahan Maintenance: Memisahkan alur transaksi ke halaman tersendiri membuat pengalaman pengguna (UX) menjadi lebih fokus dan kodenya tidak menumpuk di satu file halaman utama.
3. Direktori `lib/services` (Data Layer)
- `api_service.dart`:
 - Peran: Pusat kontrol seluruh komunikasi HTTP request (GET untuk tiket dan POST untuk transaksi) yang lari ke Backend port 3000.
 - Kemudahan Maintenance: Ini bagian yang paling krusial. Jika alamat IP server backend berubah (misalnya dari `localhost:3000` naik ke *hosting* produksi/cloud), kamu cukup mengubah satu baris basis URL di file ini saja. Seluruh halaman seperti `home_page.dart` dan `booking_page.dart` akan otomatis ikut terupdate.

22. Halaman Pemesanan & Metode Pembayaran



Berkas `booking_page.dart` mengimplementasikan halaman kasir mandiri (*Self-service Checkout*) yang adaptif, memungkinkan pelanggan memilih tiket serta menentukan metode pembayaran melalui antarmuka yang modern.

1. Desain Visual Modern (CustomScrollView & SliverAppBar)

- Sliver Architecture: Mengganti ListView biasa dengan CustomScrollView dan SliverAppBar.
 - Analisis UI/UX: SliverAppBar menggunakan efek gradasi linear dari biru ke aqua (LinearGradient). Saat halaman di-scroll ke bawah, AppBar akan mengecil dan menempel secara elegan (*pinned: true*), memberikan sisa ruang yang lega untuk daftar tiket.
- Modularitas Widget: Kode dipecah secara rapi menjadi fungsi-fungsi kecil pembangunan komponen (*Component-Driven Widgets*), seperti `_buildInputCard()`, `_buildTicketList()`, dan `_buildTicketItem()`. Ini membuat kode mudah dipelihara (*maintainable*).

2. Alur Gateway Pembayaran (Modal System with Back Button)

Ini adalah fitur paling kompleks dan canggih di aplikasi Flutter kamu. Kamu membangun sistem alur navigasi di dalam *Bottom Sheet* tanpa perlu berpindah halaman baru.

- Main Payment Modal (`_showMainPayment`):
 - Muncul ketika tombol "Pilih" pada tiket ditekan. Menampilkan opsi metode pembayaran utama: QRIS Fast Pay, E-Wallet, dan M-Banking.
- Sub-Payment Modal Navigation (`_showSubPayment`):
 - Jika pengguna memilih E-Wallet atau M-Banking, sistem akan memanggil lembaran baru yang berisi daftar vendor resmi (OVO, GoPay, Dana, BCA, Mandiri, dll).
- Stateful Back Button (`showBack: true`):
 - Menggunakan parameter logika untuk memunculkan tombol kembali (`Icons.arrow_back_ios`). Jika diklik, lembaran sub-pembayaran akan ditutup (`Navigator.pop`) dan otomatis membuka kembali menu utama (`_showMainPayment`). Ini menjamin alur UX yang mulus (*seamless payment flow*).

3. Penanganan Transaksi & Feedback Real-Time (prosesBayar)

- Asynchronous Loader: Saat proses pembayaran berlangsung, sistem memicu loading indicator menggunakan `showDialog` untuk mengunci layar sementara waktu, mencegah pengguna melakukan klik ganda yang bisa menduplikasi data di backend.
- Mounted Safety Check: Menggunakan kode `if (!mounted) return;`. Ini adalah standar coding Flutter yang baik untuk memastikan widget masih aktif di layar sebelum melakukan aksi penutupan dialog loading, sehingga mencegah aplikasi crash.
- Floating SnackBar (`_triggerSnackBar`):

- Memberikan notifikasi melayang (floating behavior) dengan sudut membulat yang informatif, memberi tahu pengguna bahwa transaksi sukses beserta metode pembayaran yang mereka pilih (contoh: "Sukses! Tiket berhasil dibayar via GoPay").